University for the Creative Arts

BSBI School of Business and Innovation

Assignment Title: Design and Report on a Big Data Analytics Solution for Business Decision-Making

Project Title: Big Data Analytics for Urban Service Optimisation: A Case Study of NYC 311 Service Requests

Programme Title: BSc Computer Science and Digitalization

Module: Big Data Analytics using AI

Name: Obianisi Leonard Tochukwu

Student ID: Q1056903

Year:2026

Lecturer:Dr Mohammed Nazeh Alimam

CONTENTS

Statement of Compliance with Academic Ethics and the Avoidance of Plagiarism

1. Introduction
1.1 Organisational Context
1.2 Problem and Decision Need
1.3 Aim and Research Questions
1.4 Scope and Analytical Approach

2. Literature Review
2.1 Big Data and the 5Vs
2.2 Hadoop Ecosystem
  2.2.1 Hadoop Distributed File System (HDFS)
  2.2.2 YARN
  2.2.3 MapReduce
2.3 Apache Spark and Distributed Analytics
2.4 Big Data Architecture and Processing Layers

3. Methodology
3.1 Problem Analysis and Analytical Use Cases
3.2 Dataset Acquisition and Initial Prototype
3.3 Data Cleaning, Transformation and Validation
3.4 Cloud Storage and Dataproc Implementation
3.5 PySpark Analysis and Metric Implementation
3.6 Performance Optimisation: CSV, Parquet and Caching
3.7 Testing, Issues and Corrective Actions

4. Results and Discussion
4.1 Complaint Demand Patterns
4.2 Borough and Agency Demand Patterns
4.3 Temporal Demand Patterns
4.4 Recorded Resolution-Time Patterns
4.5 Performance Evaluation
4.6 Decision-Support Implications and Overall Discussion

5. Conclusion

References

Appendices
Appendix A – Local Prototype and Data Preparation
Appendix B – Cloud and Dataproc Implementation

Appendix C – Performance Evaluation
Appendix D – Full-Year Analysis and Validation

LIST OF FIGURES

Figure 2.1. Layered Big Data Architecture Applied to the NYC311 Analytical Solution
Figure 3.1. Methodological Framework for the NYC311 Big Data Analysis
Figure 3.2. Python/Pandas Cleaning and Feature-Engineering Validation Output
Figure 3.3. Successful Spark Application Execution Managed through YARN on Google Cloud Dataproc
Figure 4.1. Top 10 NYC311 Complaint Categories in 2024
Figure 4.2. NYC311 Service Requests by Borough in 2024
Figure 4.3. Monthly NYC311 Service Requests in 2024
Figure 4.4. Median Recorded Resolution Time by Borough
Figure 4.5. Median Spark Execution Times for Storage Format and Caching Tests

LIST OF TABLES

Table 2.1. Application of the 5Vs to the NYC311 Dataset
Table 3.1. Data Cleaning, Transformation and Validation Procedures
Table 3.2. Analytical Metrics Implemented in the PySpark Workflow
Table 3.3. Performance Optimisation Tests
Table 3.4. Testing, Issues and Corrective Actions
Table 4.1. Leading Agencies by NYC311 Request Volume
Table 4.2. Key Temporal Demand Metrics

LIST OF ABBREVIATIONS

API – Application Programming Interface
CSV – Comma-Separated Values
GCS – Google Cloud Storage
HDFS – Hadoop Distributed File System
NYC311 – New York City 311
SQL – Structured Query Language
YARN – Yet Another Resource Negotiator

Statement of compliance with academic ethics and the avoidance of plagiarism

I honestly declare that this dissertation is entirely my own work and none of its part has been copied from printed or electronic sources, translated from foreign sources and reproduced from essays of other researchers or students. Wherever I have been based on ideas or other people texts I clearly declare it through the good use of references following academic ethics.

(In the case that is proved that part of the essay does not constitute an original work, but a copy of an already published essay or from another source, the student will be expelled permanently from the postgraduate program).

Name and Surname (Capital letters):

OBIANISI TOCHUKWU LEONARD

Date: ...10..../...09.../....2026....

1. INTRODUCTION

1.1 Organisational Context

New York City's 311 service provides residents, businesses and visitors with access to non-emergency government information and services, directing requests to the relevant City agency (NYC311, n.d.).

The associated *311 Service Requests from 2020 to Present* dataset is made publicly available through NYC Open Data and updated daily. Each record represents a service request and includes attributes such as complaint type, responding agency and geographic location (NYC Open Data, 2026). Its scale, continuous generation, multidimensional structure and data-quality considerations make it suitable for examining Big Data analytics and align with the characteristics examined further in Section 2.1.

Elgendy and Elragal (2016) argue that Big Data analytics creates organisational value when technologies, architecture and analysis are connected to decision-making rather than treated only as technical infrastructure.

1.2 Problem and Decision Need

Large volumes of service-request records do not automatically provide useful operational knowledge. Decision-makers need to identify meaningful patterns within the data, including which complaint categories generate the greatest demand, where requests are concentrated, when demand peaks and how recorded resolution times vary across the service system.

This study therefore examines how NYC311 data can be transformed into **validated, interpretable and actionable analytical metrics** that may support municipal service planning and resource allocation. High request volumes or longer recorded resolution times are not treated as direct evidence of poor performance, because these outcomes may also reflect complaint type, agency responsibilities, reporting behaviour and differences in operational processes.

Although the 2024 dataset can be analysed using conventional single-machine tools, its continuing growth, ongoing data generation and multidimensional structure make it appropriate for demonstrating how scalable Big Data technologies can support repeatable analysis beyond an initial local prototype.

1.3 Aim and Research Questions

The aim of this study is to design, implement and evaluate a scalable Big Data analytics solution for NYC311 service-request data to identify operational patterns that can support municipal service planning and resource allocation.

The primary research question is:

How can scalable Big Data analytics identify spatial, temporal and service-resolution patterns in NYC311 data to support municipal service planning?

The analysis examines complaint volume, borough and agency demand, monthly, daily and hourly patterns, and recorded resolution times. A secondary technical question evaluates how storage format and caching affect the efficiency of repeated analytical workloads.

1.4 Scope and Analytical Approach

The study focuses on service requests created during the 2024 calendar year. The workflow was first prototyped on 500,000 records using Python and Pandas. Because the records were retrieved chronologically, assessment showed that the prototype covered only 1 January to 26 February 2024 and could introduce seasonal bias. The final analytical scope was therefore expanded to the complete 2024 dataset before substantive conclusions were produced.

The workflow progresses from data acquisition and validation to cloud storage, distributed processing, analytical aggregation, performance evaluation, visualisation and decision support. Python and Pandas supported initial profiling and transformation. Google Cloud Storage and Google Cloud Dataproc supported the cloud implementation, while PySpark and Spark SQL were used for full-year processing and analysis. CSV versus Parquet storage formats and uncached versus cached Spark operations were subsequently evaluated through controlled performance comparisons.

2. Literature Review

2.1 Big Data and the 5Vs

Big Data is not defined only by the physical size of a dataset. It also concerns the speed at which data are generated, the range of data types involved, the reliability of the data and whether useful value can be extracted from them. De Mauro, Greco and Grimaldi (2016) describe Big Data as information assets characterised by high volume, velocity and variety that require appropriate technologies and analytical methods to generate value. This extends the earlier three-dimensional view of Big Data and reflects the broader **5Vs framework: Volume, Velocity, Variety, Veracity and Value**.

The framework is relevant to NYC311 because the dataset demonstrates each characteristic in an operational setting. The 2024 dataset used in this study contained **3,456,769 raw service-request records**, representing the volume dimension. Velocity is reflected in the continuous creation of new service requests and the daily updating of the public dataset. Variety is present through the combination of categorical, temporal and geographical attributes, including complaint type, agency, timestamps, borough and coordinates.

Veracity concerns whether the data are sufficiently reliable for analysis. Operational data may contain missing, inconsistent or invalid values, making data profiling, cleaning and validation necessary before analytical results can be interpreted confidently. Value is achieved when the processed records are converted into useful metrics that support understanding of service demand, workload patterns and recorded resolution times.

Table 2.1. Application of the 5Vs to the NYC311 dataset

Big Data characteristic		*Application to NYC311*
Volume		3,456,769 raw service requests analysed for the 2024 calendar year
Velocity		New service requests are generated continuously and the public dataset is updated regularly
Variety		Categorical, temporal and geographical attributes are analysed together
Veracity		Missing, invalid and inconsistent values require cleaning and validation
Value		Analytical metrics can support service planning and resource-allocation decisions

***S**ource: Author, based on De Mauro, Greco and Grimaldi (2016), NYC Open Data (2026) and the implemented NYC311 analysis.*

The 5Vs therefore provide more than a classification of dataset characteristics. In this study, they also connect the properties of NYC311 data to the need for an analytical workflow capable of managing data quality, repeated processing and the production of decision-relevant information.

2.2 Hadoop Ecosystem

Apache Hadoop is an open-source framework for distributed storage and processing across clusters of computers. Its core concepts include HDFS for storage, YARN for resource management and MapReduce as a distributed processing model.

2.2.1 Hadoop Distributed File System (HDFS)

HDFS divides large files into blocks and stores them across multiple DataNodes, while a NameNode manages file-system metadata and tracks block locations. The design supports fault tolerance and high-throughput access to large datasets (Apache Software Foundation, 2026). In this project, HDFS is discussed as Hadoop's native distributed-storage model, while Google Cloud Storage was used for persistent cloud storage.

2.2.2 YARN

YARN, or *Yet Another Resource Negotiator*, manages computational resources within a Hadoop cluster. It separates resource management from the processing framework, allowing multiple applications to share cluster resources more flexibly (Vavilapalli et al., 2013). This is relevant to the Dataproc environment used in the study, where Spark applications operated within a Hadoop-based cluster environment supported by YARN.

2.2.3 MapReduce

MapReduce is a distributed processing model in which the **Map** stage transforms input records into intermediate key-value pairs, shuffle and sort groups matching keys, and the **Reduce** stage aggregates the grouped values (Dean and Ghemawat, 2008). It was not directly programmed in the NYC311 implementation because Spark was used instead, but it remains important for understanding distributed-processing principles.

2.3 Apache Spark and Distributed Analytics

Apache Spark is a distributed processing engine designed for large-scale analytical workloads. Unlike traditional MapReduce, Spark supports iterative and interactive operations within a unified environment. Zaharia et al. (2016) emphasise Spark's ability to reuse data across processing stages, improving the efficiency of repeated analytical tasks.

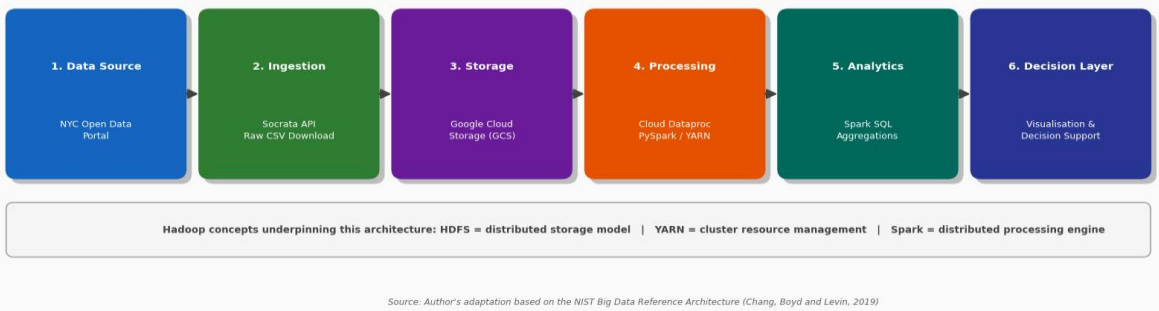
Spark DataFrames provide a structured representation of distributed data, while Spark SQL enables filtering, grouping and aggregation using SQL-style queries. In this study, PySpark applied cleaning and transformation logic to the complete 2024 dataset, while Spark SQL supported analysis by complaint type, borough, agency and time period. Spark also enabled controlled evaluation of Parquet storage and caching.

2.4 Big Data Architecture and Processing Layers

Big Data architectures separate data handling into connected functional layers. The NIST Big Data Reference Architecture describes roles and activities covering data provision, processing, analytics and consumption (Chang, Boyd and Levin, 2019).

For this study, the architecture is represented through six layers: **data source, ingestion, storage, processing, analytics, and visualisation and decision support**. NYC Open Data forms the source layer; the Socrata API and CSV files support ingestion; Google Cloud Storage provides persistent storage; Dataproc and PySpark form the processing layer; Spark SQL supports analytical aggregation; and visualisations translate the resulting metrics into decision-support information.

Figure 2.1 Layered Big Data Architecture Applied to NYC311



Source: Author's adaptation based on the NIST Big Data Reference Architecture (Chang, Boyd and Levin, 2019)

3. Methodology

3.1 Problem Analysis and Use-Case Scope

The methodological process began by defining the problem and limiting the scope of the analytical use cases. The problem was not simply the availability of a large NYC311 dataset, but how to transform service-request records into validated information that could support municipal planning.

The use cases focused on four operational questions: which complaints generate the greatest demand, where demand is concentrated, when demand peaks, and how recorded resolution times vary. A secondary technical use case examined whether storage format and Spark caching could improve repeated analytical processing.

The solution was evaluated using five criteria: **correctness, efficiency, maintainability, scalability and implementation complexity**.

Table 3.1. Solution Evaluation Criteria

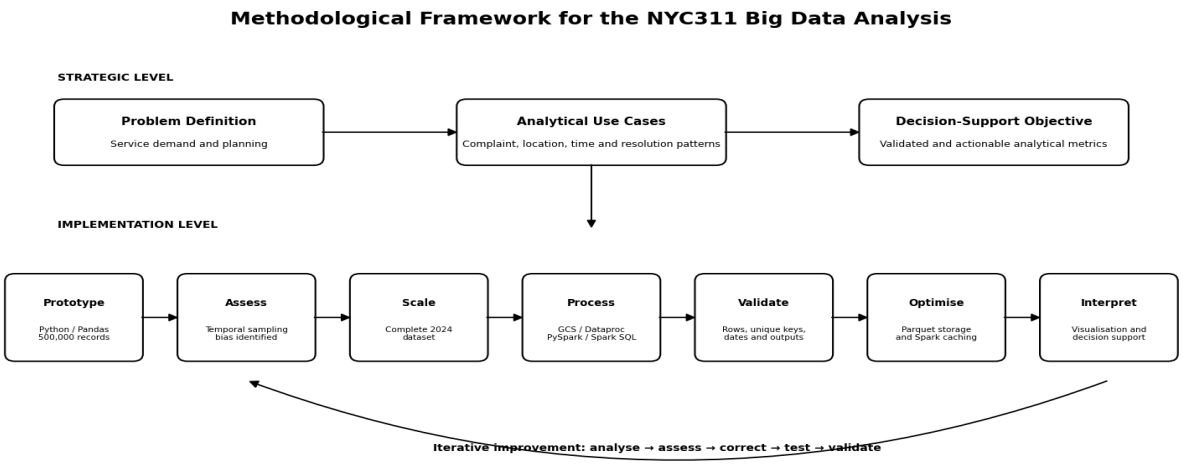
Criterion	Application in this study
Complexity	Assess whether the selected cloud architecture remains understandable and manageable
Efficiency	Compare Spark execution times
Maintainability	Use repeatable scripts and clearly separated processing stages
Scalability	Use technologies capable of distributed processing while acknowledging the single-node test environment
Correctness	Validate row counts, keys, dates and analytical outputs

3.2 Proposed Method Construct

The proposed method followed an iterative workflow rather than a single algorithm. It progressed through **problem definition, prototyping, assessment, scaling, processing, validation, optimisation and interpretation**.

An initial local prototype was used to test the analytical logic before the workflow was transferred to cloud-based processing. Problems identified during assessment were corrected before final interpretation. Figure 3.1 summarises this method and shows the relationship between the strategic objective and the implementation stages.

Figure 3.1. Methodological Framework for the NYC311 Big Data Analysis



Source: Author's methodological framework based on the implemented NYC311 analytical workflow

3.3 Data Cleaning, Transformation and Validation

Data were obtained from the NYC Open Data *311 Service Requests from 2020 to Present* dataset and restricted to requests created during 2024 (NYC Open Data, 2026). Fourteen variables were selected for analysis.

The first implementation used **500,000 records in Python and Pandas**. This stage supported data profiling, cleaning and feature engineering. Records missing borough or complaint_type were removed, timestamps were converted to datetime format, and the variables resolution_hours, hour_of_day, day_of_week and month were derived.

The prototype covered only **1 January to 26 February 2024**, revealing temporal bias. The workflow was therefore expanded to the complete annual dataset.

The full extraction contained **3,456,769 records**. Validation confirmed 3,456,769 distinct unique_key values, zero duplicate keys and a date range from **1 January to 31 December 2024**. After cleaning, **3,454,131 records** remained (see Appendix D, Figure D.1).

Figure 3.2. Python/Pandas Cleaning and Feature-Engineering Validation Output.

```
Rows before cleaning: 500,000
Rows after cleaning: 499,629
Created variables: ['resolution_hours', 'hour_of_day', 'day_of_week', 'month']
```

Source: Author's Python/Pandas implementation.

Table 3.1. Data Cleaning, Transformation and Validation Procedures

Procedure	Implementation	Purpose
Missing-value handling	Removed records missing borough or `complaint_type`	Ensure essential analytical fields were available
Borough validation	Restricted records to the five NYC boroughs	Maintain geographical consistency
Datetime conversion	Converted creation and closure timestamps	Enable temporal and resolution-time analysis
Feature engineering	Created `resolution_hours`, `hour_of_day`, `day_of_week`and `month`	Support the defined analytical use cases
Duplicate validation	Compared total records with distinct `unique_key values`	Check for duplicate service requests
Date-range validation	Confirmed the first and last timestamps	Verify complete 2024 coverage
Row-count validation	Compared expected and retrieved record totals	Confirm extraction completeness

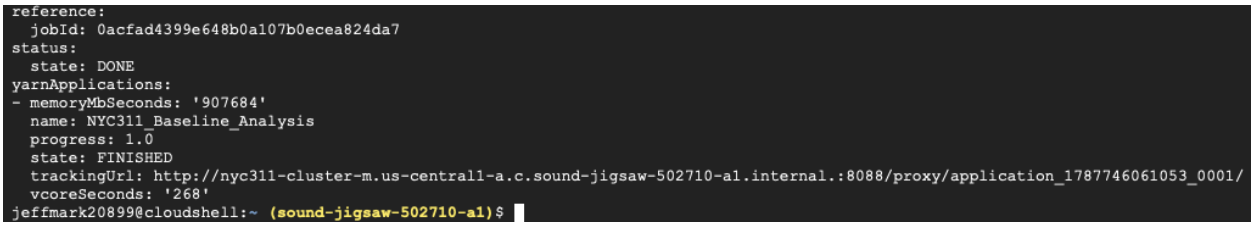
Source: Author, based on the implemented Python and PySpark workflow.

3.4 Technical Environment and Feasibility

The cloud implementation used **Google Cloud Storage** for persistent storage and **Google Cloud Dataproc** for Spark execution. PySpark and Spark SQL were used for transformation and analytical aggregation, while YARN managed the Spark application within the Dataproc environment.

The proposed workflow was technically feasible because the source data were publicly accessible, the required analytical variables were available and the selected technologies supported storage, processing and repeated analysis. However, the practical implementation used a **single-node Dataproc cluster**, so the study demonstrates the use of scalable technologies but does not claim to prove multi-node scalability.

Figure 3.3. Successful Spark Application Execution Managed through YARN on Google Cloud Dataproc



Source: Author's Google Cloud Dataproc implementation.

3.5 PySpark Analysis and Metric Implementation

PySpark and Spark SQL were used to generate the metrics required by the defined use cases.

Table 3.2. Analytical Metrics Implemented

Analytical area	Metric
Complaint demand	Request count by complaint type
Geographic demand	Request count by borough
Agency workload	Request count by agency
Temporal patterns	Monthly, daily and hourly request counts
Resolution patterns	Median resolution hours

Recorded resolution time was calculated using created_date and closed_date. Only valid non-negative values were included. The median was used because resolution times were highly variable and could be influenced by extreme observations.

3.6 Performance Optimisation: CSV, Parquet and Caching

Two controlled experiments evaluated processing efficiency.

CSV and Parquet were first compared using repeated Spark workloads. Median execution time decreased from **7.816 seconds using CSV to 1.587 seconds using Parquet**, a measured **79.69% reduction**.

Caching was then evaluated separately. Median execution time decreased from **7.098 seconds uncached to 1.162 seconds cached**, representing an **83.63% reduction after cache materialisation**. The initial cache operation required **9.111 seconds**, showing that caching is most useful when data are reused repeatedly.

Table 3.3. Performance Comparison

Test	Baseline median	Improved median	Measured change
CSV vs Parquet	7.816 s	1.587 s	79.69% reduction
Uncached vs cached	7.098 s	1.162 s	83.63% reduction

Source: Author's controlled Dataproc experiments.

These tests address **efficiency**, while the use of reusable scripts and separated processing stages supports maintainability. Scalability remains a technology-level capability rather than an experimentally verified multi-node result.

3.7 Testing Issues and Corrective Actions

The methodology included repeated comparison between the initial state and the validated final state.

Table 3.4. Initial State, Corrective Action and Improved State

Stage	Project state
Initial state	500,000 chronologically ordered records
Problem identified	Dataset covered only January–February 2024
Corrective action	Expanded extraction to the complete 2024 dataset
Validation	3,456,769 expected and actual rows; zero duplicate keys; full-year coverage
Improved state	Complete validated annual dataset used for final analysis

Source: Author, based on the implemented NYC311 workflow.

4. Results and Discussion

4.1 Complaint Demand Patterns

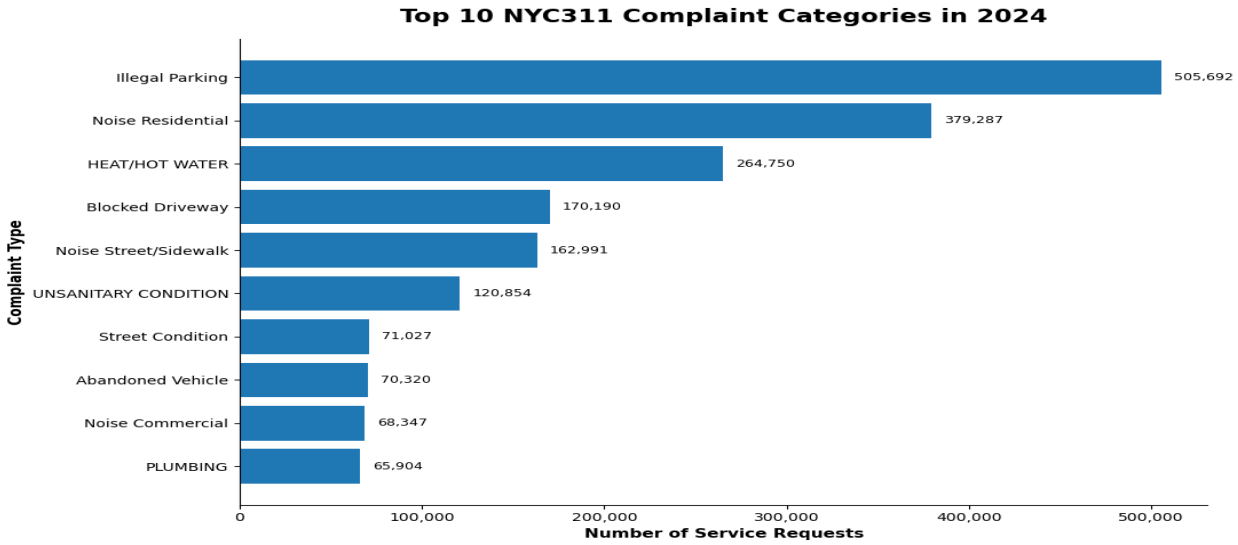
Analysis of the cleaned 2024 dataset showed clear differences in request volume across complaint categories. **Illegal Parking** was the most frequently recorded complaint, with **505,692 requests**, representing approximately **14.6%** of the 3,454,131 cleaned records. It was followed by **Noise Residential** with 379,287 requests and **HEAT/HOT WATER** with 264,750 requests.

This full-year result differed from the initial 500,000-record prototype, in which **HEAT/HOT WATER** was the leading complaint. The prototype covered only January to February and therefore over-represented winter-related demand. After the analysis was expanded to the complete year, Illegal Parking became the largest category. This change shows why validating the temporal coverage of the data was necessary before interpreting service-demand patterns.

The results identify categories that generate substantial reported demand, but request counts should not be treated as direct measures of the underlying incidence of each problem. Reporting behaviour, service awareness and access to NYC311 may also influence the observed totals.

Raw PySpark evidence for the full-year complaint output is provided in **Appendix D, Figure D.3**.

Figure 4.1 Top 10 NYC311 Complaint Categories in 2024



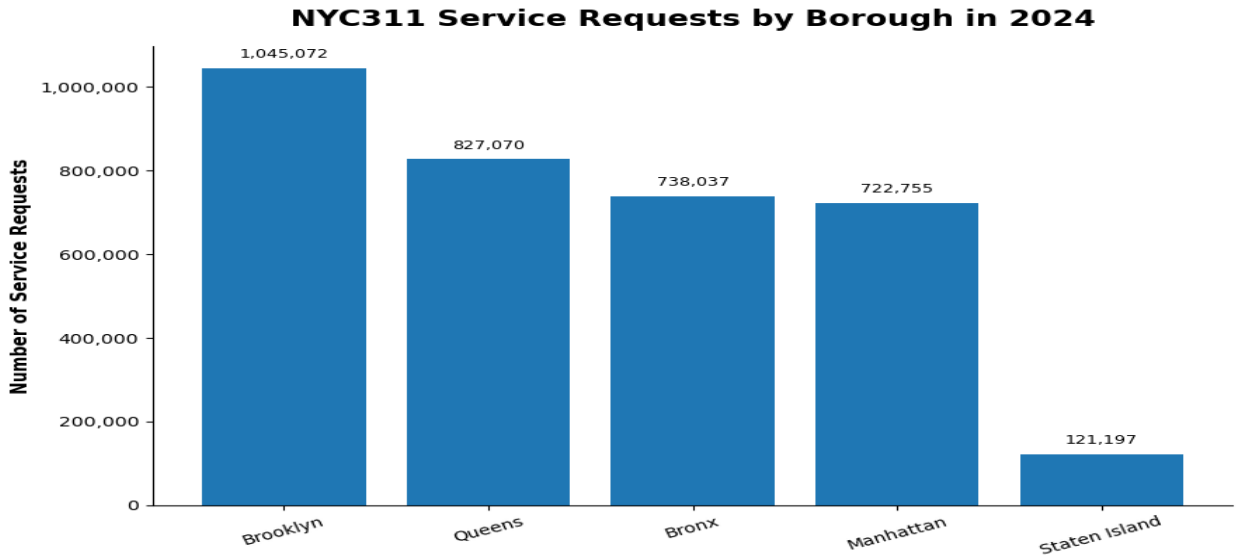
Source: Author's analysis of NYC Open Data.

4.2 Borough and Agency Demand Patterns

Request volume also varied geographically. **Brooklyn recorded 1,045,072 requests**, equivalent to approximately **30.3%** of the cleaned dataset, followed by Queens with 827,070, the Bronx with 738,037, Manhattan with 722,755 and Staten Island with 121,197.

These are absolute counts rather than population-adjusted rates. The results therefore indicate where recorded service demand was concentrated, but they do not establish that one borough experienced more underlying problems than another. Population size, reporting behavior and complaint composition may influence the differences.

Figure 4.2 NYC311 Service Requests by Borough in 2024



Source: Author's analysis of NYC Open Data.

Agency workload showed an even stronger concentration. **NYPD received 1,547,995 requests**, approximately **44.8%** of cleaned records. HPD followed with 733,927 requests and DSNY with 305,916. These totals reflect the categories routed to each agency and should not be interpreted as measures of organisational performance.

Table 4.1. Leading Agencies by NYC311 Request Volume

Agency	Number of requests
NYPD	1,547,995
HPD	733,927
DSNY	305,916
DOT	203,679
DEP	191,646
DPR	138,328

DOB	102,783
DOHMH	90,369
DHS	51,983
TLC	37,134

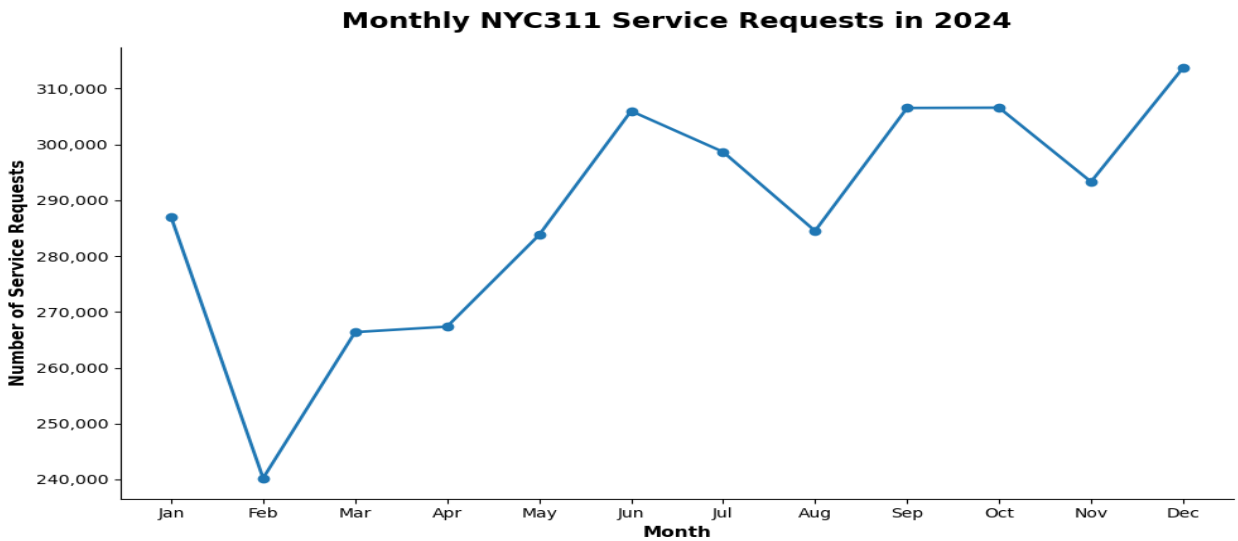
Source: Author's analysis of NYC Open Data.

For decision support, borough and agency aggregations are most useful as indicators of where demand enters the service system and which organisations receive the largest recorded workloads.

4.3 Temporal Demand Patterns

Service-request activity changed across the year, week and day. **December recorded the highest monthly volume with 313,735 requests**, while **February recorded the lowest with 240,228**. September and October also exceeded 306,000 requests.

Figure 4.3 Monthly NYC311 Service Requests in 2024



At day-of-week level, **Monday recorded the highest volume with 541,823 requests**, while Saturday

recorded the lowest with 438,205. At hourly level, the highest volume occurred at **10:00, with 208,438 requests**.

Source: Author's analysis of NYC Open Data.

Table 4.2. Key Temporal Demand Metrics

Temporal measure	Highest observed period	Requests
Month	December	313,735
Day of week	Monday	541,823
Hour of day	10:00	208,438

Source: Author's analysis of NYC Open Data.

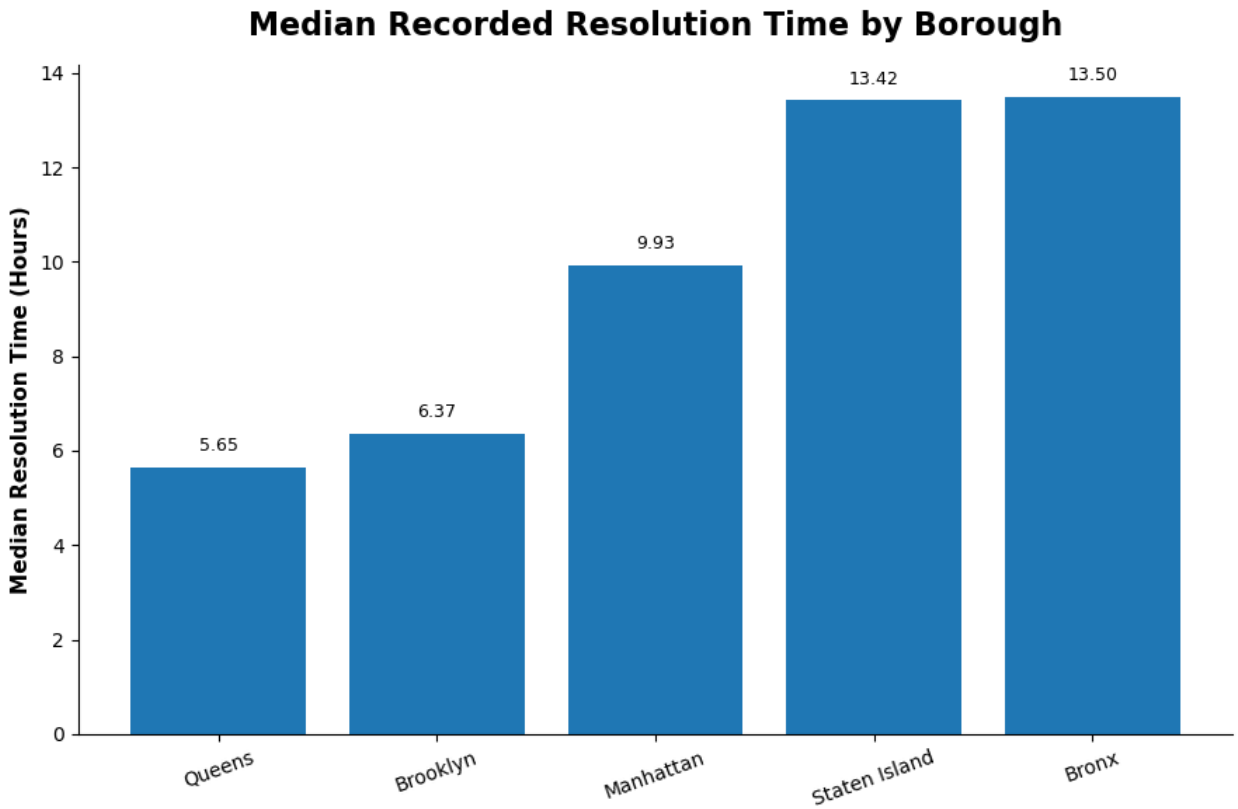
The annual analysis again differed from the initial prototype. The 500,000-record sample showed a **09:00 peak**, whereas the complete 2024 dataset showed a **10:00 peak**. Together with the change in the leading complaint category, this illustrates how conclusions can change when a temporally restricted development sample is replaced with complete annual coverage.

These patterns may support staffing, monitoring and workload planning during higher-volume periods. However, the analysis identifies **when** requests were recorded, not why demand increased. Seasonality, complaint mix, reporting behaviour and service availability may all contribute to the observed variation.

4.4 Recorded Resolution-Time Patterns

Recorded resolution times also varied across boroughs. The median was lowest in **Queens at 5.65 hours**, followed by Brooklyn at 6.37 hours and Manhattan at 9.93 hours. Staten Island recorded a median of 13.42 hours and the Bronx 13.50 hours.

Figure 4.4 Median Recorded Resolution Time by Borough



Source: Author's analysis of NYC Open Data.

The median was used because resolution-time values were highly variable and could be distorted by extreme observations. Agency-level analysis, restricted to agencies with at least **1,000 valid resolved requests**, showed an even wider range. Median recorded resolution time ranged from approximately **1.06 hours for NYPD to 1,742.10 hours for TLC**.

These values should not be interpreted as a ranking of agency or borough performance. Agencies handle different complaint categories, operate under different procedures and may apply different closure practices. The results instead identify where recorded resolution patterns differ and where further operational investigation may be useful.

4.5 Performance Evaluation

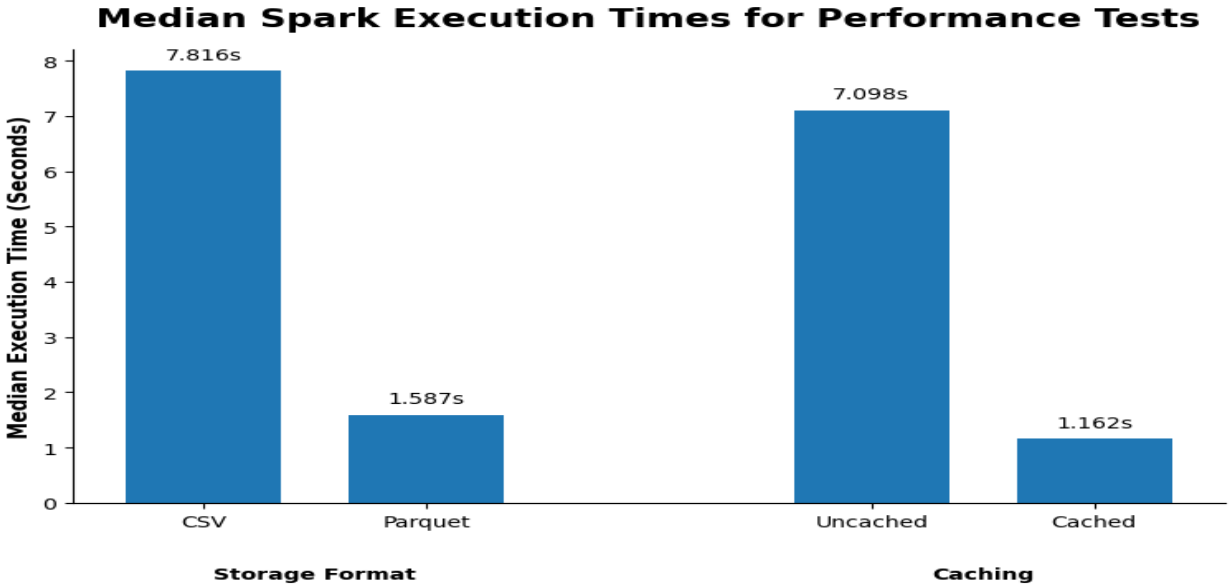
The technical experiments showed that storage format and caching affected repeated Spark execution time within the implemented Dataproc environment.

For the storage-format test, median execution time was **7.816 seconds using CSV** and **1.587 seconds using Parquet**. This represented a measured **79.69% reduction in median execution time**. Parquet's

column-oriented structure is suited to analytical workloads because Spark can access required columns without repeatedly processing complete text-based records.

In the separate caching experiment, median execution time decreased from **7.098 seconds for uncached operations to 1.162 seconds after caching**, representing an **83.63% reduction**. However, materialising the cache required **9.111 seconds**. Caching therefore provides the greatest benefit when the same dataset is reused across several operations rather than for a single query.

Figure 4.5 Median Spark Execution Times for Storage Format and Caching Tests.



Source: Author's controlled Dataproc performance experiments.

The results should be interpreted within the limits of the test environment. The experiments were conducted on a single-node Dataproc configuration and do not establish universal performance gains for all datasets or cluster sizes. Instead, they demonstrate that technical design choices can substantially affect repeated analytical workloads and therefore form part of an effective Big Data solution.

4.6 Decision-Support Implications and Overall Discussion

Taken together, the results show how a Big Data analytics workflow using scalable technologies can transform a large operational dataset into decision-relevant metrics. Complaint, borough, agency and temporal aggregations identify where and when recorded demand is concentrated, while resolution-time analysis highlights variation in service-processing patterns.

This supports the primary research question by demonstrating how **spatial, temporal and service-resolution pattern** Scan be extracted from NYC311 data to inform service planning. The results do not prescribe specific staffing levels or resource allocations; rather, they provide evidence that decision-makers could combine with operational knowledge, population information and service priorities.

The technical findings address the secondary research question. In the implemented environment, **Parquet storage and caching reduced median execution times for repeated Spark workloads**, showing that storage and processing choices affect analytical efficiency as well as the outputs produced.

This relationship between analytics and organisational use is consistent with Elgendy and Elragal (2016), who argue that Big Data creates value when analytical capabilities are connected to decision-making. In this project, that connection is represented by the movement from validated service-request records to interpretable demand, timing, resolution and performance metrics.

5. Conclusion

This study designed and implemented a Big Data analytics solution using NYC311 service-request data to identify spatial, temporal and service-resolution patterns that may support municipal service planning. The final analysis used the complete 2024 dataset containing **3,456,769 raw records**, of which **3,454,131 records** remained after cleaning and validation.

The analysis identified clear operational patterns. **Illegal Parking** generated the highest complaint volume, **Brooklyn** recorded the largest absolute borough-level demand, and **NYPD** received the greatest number of service requests. Temporal analysis showed that demand was highest in **December**, on **Mondays** and at approximately **10:00**. Recorded resolution times also varied across boroughs and agencies, although these differences were treated as operational patterns rather than direct measures of organisational performance.

A key methodological finding was the importance of validation. The initial 500,000-record prototype covered only January to February 2024 and produced different results from the complete annual dataset. Expanding the analysis to the full year demonstrated how incomplete temporal coverage can affect analytical conclusions.

The technical evaluation also showed that implementation choices influenced repeated Spark workloads. Within the controlled single-node Dataproc environment, Parquet reduced median execution time by **79.69%** compared with CSV, while caching reduced repeated-operation time by **83.63%** after cache materialisation.

Overall, the study demonstrates how Python, Google Cloud Storage, Dataproc, PySpark, Spark SQL and YARN can be combined to transform large operational data into validated and interpretable metrics. The findings can support further investigation and planning, but they should be considered alongside factors such as population, complaint complexity, reporting behaviour and agency-specific operational processes. Future work could extend the analysis through population-adjusted borough comparisons, complaint-specific resolution analysis or multi-node performance testing

References

NYC311 (n.d.) *About NYC311*. Available at: [official NYC311 page] (Accessed: 17 September 2026).

Apache Software Foundation (2026) *HDFS Architecture*. Apache Hadoop 3.5.0. At: https://hadoop.apache.org/docs/current3/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html (Accessed: 17/09/2026).

Apache Software Foundation (2026) *Spark SQL, DataFrames and Datasets Guide*. Apache Spark 4.2.0 Documentation. At: https://spark.apache.org/docs/latest/sql-programming-guide (Accessed: 17/09/2026).

Chang, W.L., Boyd, D. and Levin, O. (2019) *NIST Big Data Interoperability Framework: Volume 6, Reference Architecture*. NIST Special Publication 1500-6r2. Gaithersburg, MD: National Institute of Standards and Technology. At: https://doi.org/10.6028/NIST.SP.1500-6r2 (Accessed: 17/09/2026).

Dean, J. and Ghemawat, S. (2008) 'MapReduce: Simplified Data Processing on Large Clusters', *Communications of the ACM*, 51(1), pp. 107–113. At: https://doi.org/10.1145/1327452.1327492 (Accessed: 17/09/2026).

De Mauro, A., Greco, M. and Grimaldi, M. (2016) 'A formal definition of Big Data based on its essential features', *Library Review*, 65(3), pp. 122–135. At: https://doi.org/10.1108/LR-06-2015-0061 (Accessed: 17/09/2026).

Elgendy, N. and Elragal, A. (2016) 'Big Data Analytics in Support of the Decision Making Process', *Procedia Computer Science*, 100, pp. 1071–1084. At: https://doi.org/10.1016/j.procs.2016.09.251 (Accessed: 17/09/2026).

NYC Open Data (2026) *311 Service Requests from 2020 to Present*. Dataset identifier: erm2-nwe9. At: https://data.cityofnewyork.us/Social-Services/311-Service-Requests-from-2020-to-Present/erm2-nwe9 (Accessed: 17/09/2026).

Vavilapalli, V.K., Murthy, A.C., Douglas, C., Agarwal, S., Konar, M., Evans, R., Graves, T., Lowe, J., Shah, H., Seth, S., Saha, B., Curino, C., O'Malley, O., Radia, S., Reed, B.C. and Baldeschwieler, E. (2013) 'Apache Hadoop YARN: Yet Another Resource Negotiator', *Proceedings of the 4th Annual Symposium on Cloud Computing*, pp. 5:1–5:16. New York: ACM. At: https://doi.org/10.1145/2523616.2523633 (Accessed: 17/09/2026).

Zaharia, M., Xin, R.S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M.J., Ghodsi, A., Gonzalez, J.E., Shenker, S. and Stoica, I. (2016) 'Apache Spark: A Unified Engine for Big Data Processing', *Communications of the ACM*, 59(11), pp. 56–65. At: https://doi.org/10.1145/2934664 (Accessed: 17/09/2026).

APPENDIX

Appendix A – Local Prototype and Data Preparation

Figure A.1. Initial 500,000-Record Prototype and Date-Range Validation.

```
[1]: import pandas as pd

file_path = "data/nyc311_2024_500k_raw.csv"

df = pd.read_csv(file_path, low_memory=False)

df["created_date"] = pd.to_datetime(
    df["created_date"],
    errors="coerce"
)

print("INITIAL PROTOTYPE VALIDATION")
print("----------------------------")
print(f"Total rows: {len(df):,}")
print(f"First created date: {df['created_date'].min()}")
print(f"Last created date:  {df['created_date'].max()}")
```

```
INITIAL PROTOTYPE VALIDATION
----------------------------
Total rows: 500,000
First created date: 2024-01-01 00:00:00
Last created date:  2024-02-26 20:17:07
```

Figure A.2. Python/Pandas Data Cleaning and Feature-Engineering Implementation

```
clean["hour_of_day"] = clean["created_date"].dt.hour
clean["day_of_week"] = clean["created_date"].dt.day_name()
clean["month"] = clean["created_date"].dt.month_name()

# Validation summary
print("DATA CLEANING AND FEATURE ENGINEERING")
print("-------------------------------------")
print(f"Rows before cleaning: {rows_before:,}")
print(f"Rows after cleaning:  {len(clean):,}")
print(f"Rows removed:         {rows_before - len(clean):,}")

print("\nNew analytical variables:")
print("resolution_hours")
print("hour_of_day")
print("day_of_week")
print("month")
```

```
DATA CLEANING AND FEATURE ENGINEERING
-------------------------------------
Rows before cleaning: 500,000
Rows after cleaning:  499,629
Rows removed:         371

New analytical variables:
resolution_hours
hour_of_day
day_of_week
month
```

Figure A.3. Leading Complaint Categories in the Initial 500,000-Record Prototype

```
[3]: top_complaints_500k = (
         clean["complaint_type"]
         .value_counts()
         .head(10)
         .rename_axis("Complaint Type")
         .reset_index(name="Number of Requests")
     )

     top_complaints_500k
```

[3]:

	Complaint Type	Number of Requests
0	HEAT/HOT WATER	78837
1	Illegal Parking	75458
2	Noise - Residential	41544
3	Blocked Driveway	27112
4	UNSANITARY CONDITION	18082
5	PLUMBING	11413
6	PAINT/PLASTER	11118
7	Abandoned Vehicle	10789
8	Street Condition	9889
9	WATER LEAK	8805

Appendix B- Cloud and Dataproc Implementation

Figure B.1. Google Cloud Storage Environment Used for the NYC311 Big Data Workflow

```
jeffmark20899@cloudshell:~ (sound-jigsaw-502710-a1)$ gcloud storage ls --recursive gs://sound-jigsaw-502710-a1-nyc311/ | head -n 60
gs://sound-jigsaw-502710-a1-nyc311/:

gs://sound-jigsaw-502710-a1-nyc311/data/:

gs://sound-jigsaw-502710-a1-nyc311/data/full2024/:

gs://sound-jigsaw-502710-a1-nyc311/data/full2024/processed/:

gs://sound-jigsaw-502710-a1-nyc311/data/full2024/processed/clean_parquet/:
gs://sound-jigsaw-502710-a1-nyc311/data/full2024/processed/clean_parquet/
gs://sound-jigsaw-502710-a1-nyc311/data/full2024/processed/clean_parquet/_SUCCESS
```

Figure B.2. Successful Execution of the NYC311 PySpark Job on Google Cloud Dataproc

```
jeffmark20899@cloudshell:~ (sound-jigsaw-502710-a1)$ gcloud dataproc jobs describe \
0acfad4399e648b0a107b0ecea824da7 \
--region=us-central1 \
--project=sound-jigsaw-502710-a1
done: true
```

Figure B.3 YARN Application Evidence for the NYC311 Spark Workload

```
yarnApplications:
- memoryMbSeconds: '907684'
  name: NYC311_Baseline_Analysis
  progress: 1.0
  state: FINISHED
  trackingUrl: http://nyc311-cluster-m.us-central1-a.c.sound-jigsaw-502710-a1.internal.:8088/proxy/application_1787746061053_0001/
  vcoreSeconds: '268'
jeffmark20899@cloudshell:~ (sound-jigsaw-502710-a1)$
```

Appendix C – Performance Evaluation

Figure C.1. Spark Execution-Time Results for CSV and Parquet Storage Formats

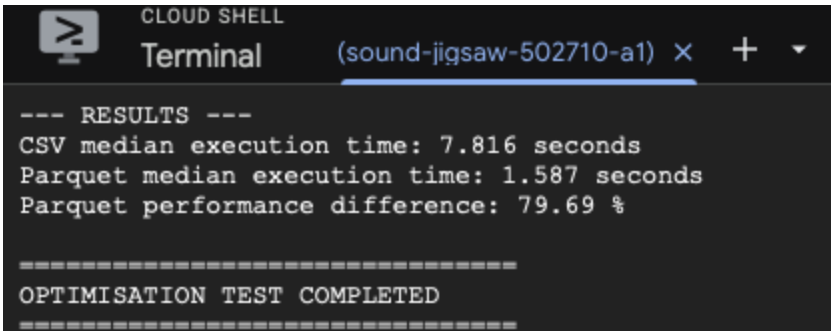
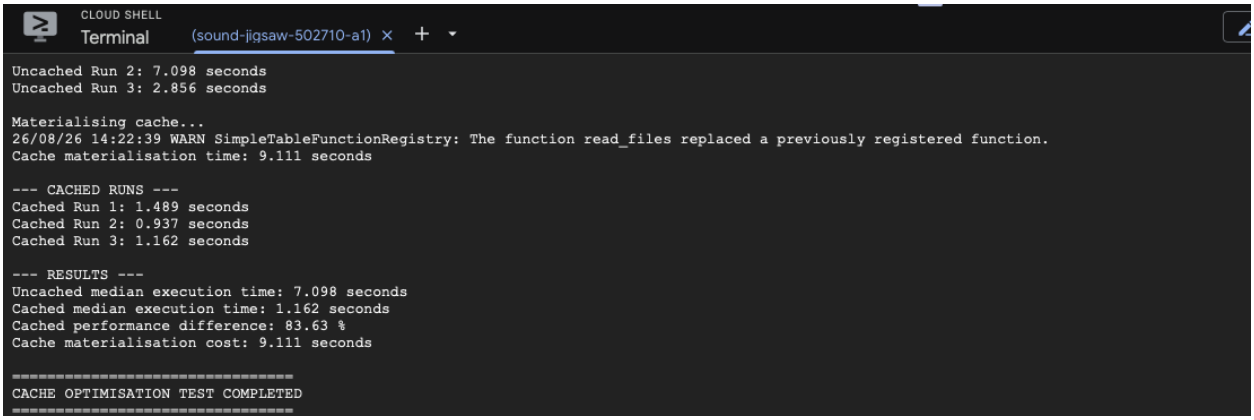


Figure C.2. Spark Execution-Time Results for Uncached and Cached Analytical Workloads



Appendix D – Full-Year Analysis and Validation

Figure D.1. Validation of the Complete 2024 NYC311 Dataset

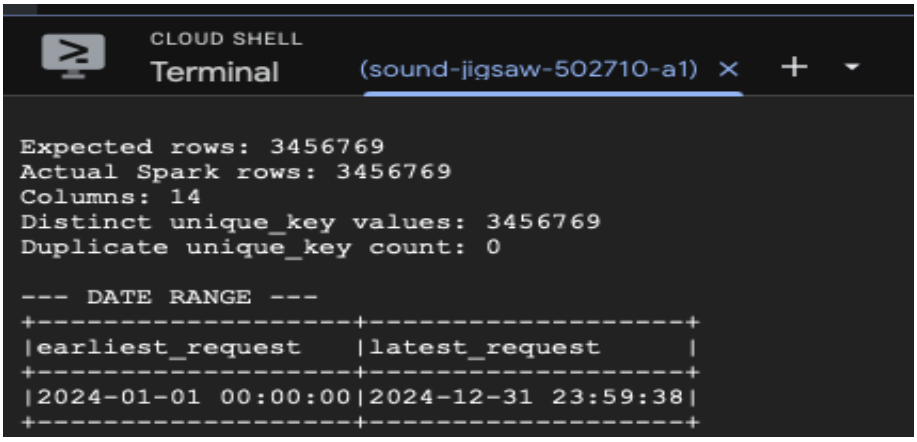


Figure D.2. Successful Completion of the Full-Year NYC311 PySpark Analysis

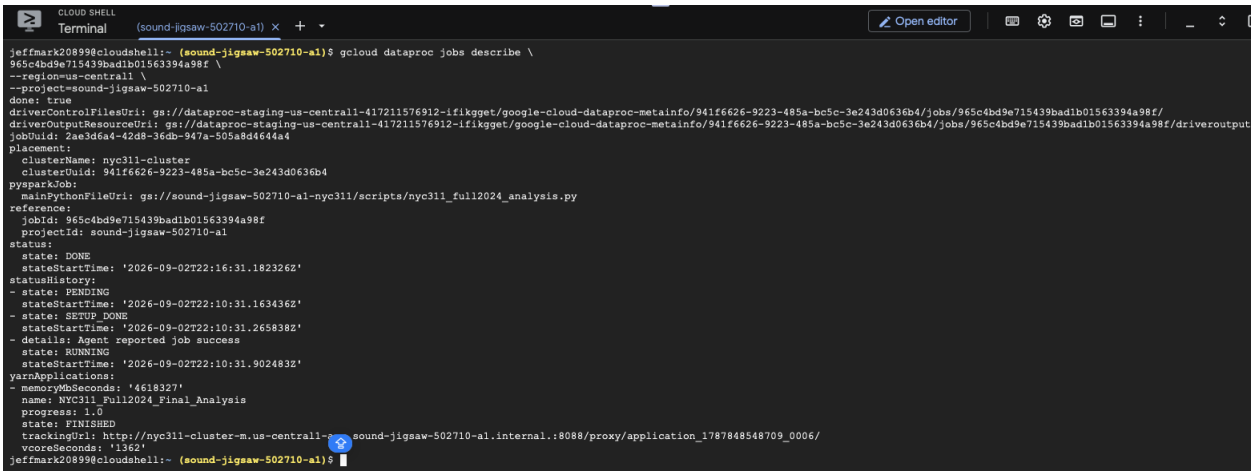


Figure D.3. Leading Complaint Categories in the Complete 2024 NYC311 Dataset

